

The mathematics of ranking_cells.py

Exact ranking probabilities of a continuous voter population on every pixel

1. Problem

The political space is the unit square $[0, 1]^2$. There are C candidates at fixed points $c_1, \dots, c_C \in [0, 1]^2$. The square is divided into $n \times n$ pixels (PIXELS), and each pixel represents a population of voters, not a single voter.

A voter at position p ranks the candidates by Euclidean distance: the closest candidate is ranked first, the next closest second, and so on. For each pixel we want

$$P_{kl}(r) = \text{share of the voters of pixel } (k, l) \text{ whose ranking is } r \quad (1)$$

for every strict ranking r that can occur. These numbers feed the voting methods (Borda, Schulze, ...) in `methods.py`. The module computes them **without sampling or discretising the voters**: the only approximations are Gauss–Legendre quadrature and polynomial interpolation, both of which converge exponentially fast.

2. Voter distribution of a pixel

2.1. Model

The voters of pixel (k, l) have positions (X, Y) with X and Y independent,

$$X \sim \text{Beta}(a_k, b_k), \quad Y \sim \text{Beta}(a_l, b_l). \quad (2)$$

The Beta family is used because it lives on $[0, 1]$, so every voter stays inside the political space. Parameters (a, b) are chosen per coordinate so that

1. the **median** is the pixel centre $m = (k + \frac{1}{2})/n$ (`pixel_medians`), and
2. the **mean absolute deviation from the median** is a fixed constant d (`DEVIATION`): $E|X - m| = d$.

Condition 2 keeps the “spread” of every pixel the same, independently of where the pixel lies. Near the edges of the square this forces $a < 1$ or $b < 1$, so the density is singular (infinite) at 0 or 1; much of the later numerics is shaped by this.

We write f, F for the pdf and CDF of X and g, G for those of Y . The CDF of $\text{Beta}(a, b)$ is the regularised incomplete beta function $I_x(a, b)$ (`scipy.special.betainc`), its inverse is `betaincinv`.

2.2. The two equations (`_solve_beta`)

The median condition is simply

$$I_m(a, b) = \frac{1}{2}. \quad (3)$$

For the absolute deviation, split at the median and use $F(m) = \frac{1}{2}$:

$$\begin{aligned} E|X - m| &= E[X - m] - 2E[(X - m)\mathbb{1}\{X < m\}] \\ &= \mu - m - 2E[X\mathbb{1}\{X < m\}] + 2mF(m) \\ &= \mu - 2E[X\mathbb{1}\{X < m\}], \end{aligned} \quad (4)$$

where $\mu = \frac{a}{a+b}$. Since $x \cdot x^{a-1}(1-x)^{b-1}/B(a, b) = \mu \cdot (\text{density of } \text{Beta}(a+1, b))$,

$$E[X\mathbb{1}\{X < m\}] = \mu I_m(a+1, b), \quad (5)$$

and therefore

$$E|X - m| = \frac{a}{a+b}(1 - 2I_m(a+1, b)) = d. \quad (6)$$

Equation 3 and Equation 6 are two equations in the two unknowns (a, b) and are solved with Levenberg-Marquardt (`scipy.optimize.root(method="lm")`).

2.3. Symmetry and continuation (`beta_params_at`)

If $X \sim \text{Beta}(a, b)$ then $1 - X \sim \text{Beta}(b, a)$. A median $m < \frac{1}{2}$ is therefore the mirror of median $1 - m$ with (a, b) swapped, so only the medians $\max(m, 1 - m) \geq \frac{1}{2}$ are solved. They are solved in increasing order, starting from $(1, 1)$ (the uniform distribution, median $\frac{1}{2}$) and using each solution as the initial guess for the next median. This continuation keeps the solver on the correct branch even far from the centre, where (a, b) change quickly.

3. Ranking cells (`ranking_cells`)

3.1. Bisectors

A voter at p prefers c_i to c_j exactly when

$$|p - c_i|^2 < |p - c_j|^2 \iff 2(c_j - c_i) \cdot p < |c_j|^2 - |c_i|^2. \quad (7)$$

The boundary of Equation 7 is the perpendicular bisector of $c_i c_j$, a straight line. A voter's full ranking is determined by the signs of all $\binom{C}{2}$ comparisons, so the ranking can only change when the voter crosses one of these bisectors.

The $\binom{C}{2}$ lines cut the unit square into convex polygons ("cells"). Inside a cell every comparison has a fixed sign, hence **every point of a cell has the same ranking**. Points on a bisector have a tie, but lines have zero area and so zero probability, and ties are ignored.

The crucial observation is that **the cells do not depend on the pixel**: they depend only on the candidates. Only the probability mass that each pixel's distribution puts on each cell changes. So

$$P_{kl}(r) = P_{kl}((X, Y) \in K_r) = \iint_{K_r} f_k(x) g_l(y) dx dy, \quad (8)$$

where K_r is the cell with ranking r .

3.2. Construction

The construction starts with the square as a single polygon and, for every pair (i, j) , splits every current polygon into its parts on either side of the bisector (half-plane clipping, `_clip`, a Sutherland-Hodgman step for one line). Slivers with area below 10^{-14} are discarded. All polygons stay convex and counter-clockwise (CCW). The ranking of a cell is found by sorting the candidates by distance from the cell's vertex average, which lies strictly inside the convex cell.

Figure 1 shows the arrangement for the candidates in `const.py`.

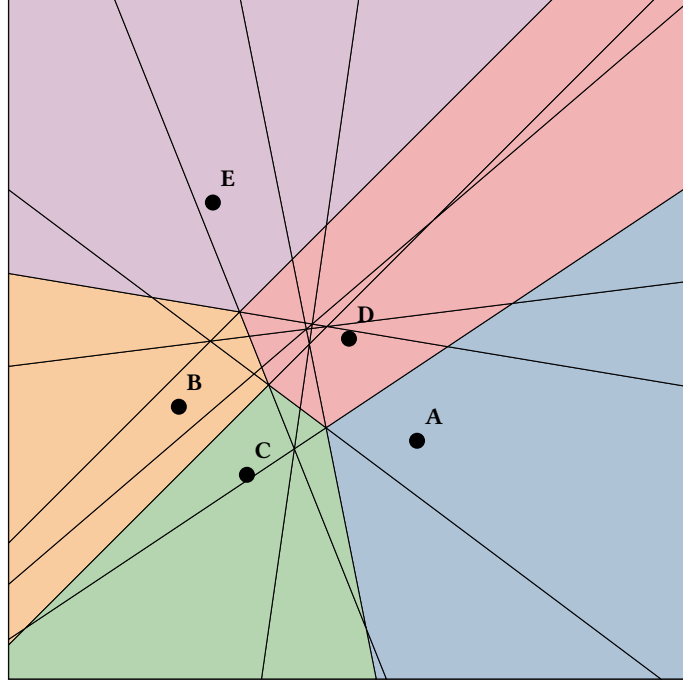


Figure 1: The 43 ranking cells of the default candidates A–E (rows of CANDIDATES). Colour marks the first choice; every cell has its own full ranking.

4. Cell probability as a boundary integral

Computing the double integral Equation 8 directly for every cell and every pixel pair would be expensive. Green’s theorem turns it into a sum of integrals along the cell’s edges, and each edge is shared by two cells.

4.1. Green’s theorem

For a region K with CCW boundary ∂K and a 1-form $\omega = L dx + M dy$,

$$\oint_{\partial K} L dx + M dy = \iint_K (\partial_x M - \partial_y L) dx dy. \quad (9)$$

Choose

$$\omega = -f(x)G(y) dx. \quad (10)$$

Then $\partial_x M - \partial_y L = f(x)g(y)$, which is the joint density, so

$$P(K) = \oint_{\partial K} \omega = \sum_{\text{edges } s \rightarrow e} \int_s^e \omega. \quad (11)$$

An equally valid choice is

$$\omega' = F(x)g(y) dy, \quad \omega' - \omega = Fg dy + fG dx = d(FG). \quad (12)$$

Because ω' and ω differ by an exact form, on any segment

$$\int_s^e \omega = \int_s^e \omega' - [F(x)G(y)]_s^e. \quad (13)$$

Which form is integrated numerically is chosen edge by edge (Section 5).

Sanity check. The probabilities of all cells sum to one: interior edges are traversed once in each direction and cancel, leaving the boundary of the square. On it $dx = 0$ on the vertical sides, $G(0) = 0$ on the bottom, and the top ($y = 1$, traversed from $x = 1$ to $x = 0$) gives $-\int_1^0 f(x) dx = 1$.

4.2. Sharing edges (compute_ranking_probabilities)

Every edge is stored once under a canonical orientation (lexicographically smaller endpoint first). A cell that traverses the edge in the canonical direction adds its integral with sign +1; the neighbouring cell traverses it the other way and adds it with sign -1 . The edges are integrated in parallel threads (betainc / betaincinv release the GIL).

The result of each edge integral is a matrix: entry $[k, l]$ uses the X -parameters of pixel column k and the Y -parameters of pixel row l . Because X and Y are independent, one set of n parameter pairs serves both axes, and the whole $n \times n$ grid of pixels comes out of $O(n)$ one-dimensional quantities combined in an outer-product fashion.

5. Edge integrals (_edge_integral)

Let the edge go from $s = (x_s, y_s)$ to $e = (x_e, y_e)$.

5.1. Axis-parallel edges (closed form)

- **Vertical** ($x_s = x_e$): $dx = 0$, so $\int \omega = 0$.
- **Horizontal** ($y_s = y_e$): $G(y)$ is constant, so

$$\int_s^e \omega = -G(y_s)(F(x_e) - F(x_s)). \quad (14)$$

These include all edges on the boundary of the square.

5.2. General edges: substitution $u = F(x)$

On the line through s and e , $y = \ell(x) = y_s + (x - x_s)(y_e - y_s)/(x_e - x_s)$. Integrating Equation 10 directly is a bad idea: $f(x)$ is infinite at 0 or 1 when $a < 1$ or $b < 1$, and quadrature would converge slowly. Substituting $u = F(x)$, $du = f(x) dx$ absorbs the density:

$$\int_s^e \omega = - \int_{F(x_s)}^{F(x_e)} G(\ell(F^{-1}(u))) du. \quad (15)$$

The integrand is now bounded by 1 and smooth unless the edge reaches $y = 0$ or $y = 1$, where G itself behaves like y^a or $1 - (1 - y)^b$ and is not smooth.

5.3. Edges touching $y = 0$ or $y = 1$: substitution $v = G(y)$

For those edges the roles of the axes are swapped. Integrate ω' from Equation 12 with $v = G(y)$, $dv = g(y) dy$, and $x = \ell^{-1}(y)$:

$$\int_s^e \omega' = \int_{G(y_s)}^{G(y_e)} F(\ell^{-1}(G^{-1}(v))) dv, \quad (16)$$

and convert back to ω with Equation 13. Now the endpoint on $y \in \{0, 1\}$ is harmless because it is mapped to $v \in \{0, 1\}$ and the singular behaviour of G is absorbed by the substitution, while x stays in the interior of $(0, 1)$ so F is smooth there.

5.4. Edges touching both kinds of boundary

An edge can run from a vertical side of the square ($x \in \{0, 1\}$) to a horizontal one ($y \in \{0, 1\}$). Neither substitution handles both endpoints, so the edge is split at its midpoint; each half touches only one kind of boundary and is handled as above. (An edge ending exactly at a corner of the square is integrated with the v form.)

5.5. Quadrature

The remaining one-dimensional integrals over $[u_s, u_e]$ (or $[v_s, v_e]$) are computed with Q -point Gauss–Legendre quadrature (QUAD_NODES, default $Q = 24$):

$$\int_{u_s}^{u_e} h(u) du \approx \frac{u_e - u_s}{2} \sum_{q=1}^Q w_q h\left(u_s + (u_e - u_s) \frac{t_q + 1}{2}\right), \quad (17)$$

with Legendre nodes t_q and weights w_q on $[-1, 1]$. Note that the interval $[F_k(x_s), F_k(x_e)]$ depends on the pixel column k , so the nodes are placed per pixel.

Cost. For the u form, F_k^{-1} is evaluated at Q nodes for each of the n column-parameters (Qn calls of `betaincinv`), then G_l is evaluated at each resulting y for each of the n row-parameters (Qn^2 calls of `betainc`). An edge therefore costs $O(Qn^2)$ Beta CDF evaluations, and the total cost is $O(E \cdot Q \cdot n^2)$ for E distinct edges.

6. Interpolation in the pixel median

With $n = \text{PIXELS}$ large, $O(Qn^2)$ per edge is still costly. But the cell probabilities are **smooth** functions of the medians (m_x, m_y) : (a, b) depends smoothly on m , and the integrals Equation 8 depend smoothly on (a, b) . (The **winner** of a voting method jumps between pixels, but the probabilities do not.) So they are computed exactly only on an $N \times N$ grid of medians (`NODES`, default $N = 49$) and interpolated to all $n \times n$ pixels. If $N \geq n$ no interpolation is used.

6.1. Nodes (`node_medians`)

The interpolation variable is $z = \text{logit}(m) = \log(m/(1 - m))$. It stretches the regions near 0 and 1, where (a, b) change fastest. The nodes are the Chebyshev–Lobatto points in z spanning the first and last pixel median:

$$t_j = -\cos\left(\pi \frac{j}{N-1}\right), \quad z_j = Z t_j, \quad m_j = \text{expit}(z_j), \quad j = 0, \dots, N-1, \quad (18)$$

where $Z = \text{logit}(1 - \frac{1}{2n})$. (The code writes t_j as $\sin(\pi \frac{2j-(N-1)}{2(N-1)})$, which is the same thing.) Chebyshev–Lobatto points cluster at the ends of the interval, which further concentrates nodes near the edges of the square and avoids the Runge phenomenon.

6.2. Barycentric interpolation (`_interpolation_matrix`)

For Chebyshev–Lobatto nodes the barycentric formula of the second kind is

$$p(z) = \frac{\sum_j \frac{\lambda_j}{z - z_j} h_j}{\sum_j \frac{\lambda_j}{z - z_j}}, \quad \lambda_j = (-1)^j, \quad \lambda_0, \lambda_{N-1} \text{ halved}, \quad (19)$$

where h_j are the values at the nodes. (The weights are invariant under the affine map $t \rightarrow Zt$, up to a common factor that cancels.) When a target coincides with a node, the node value is used directly. Evaluated at all n pixel medians this is a fixed $n \times N$ matrix L , and on the 2-D tensor grid

$$P_{\text{pixels}}[\cdot, \cdot, r] = L P_{\text{nodes}}[\cdot, \cdot, r] L^T, \quad (20)$$

applied as two matrix products (`interpolate_to_pixels`). The final product is carried out in `float32` because the result is by far the largest array; the rounding ($\approx 10^{-7}$) can only affect pixels whose winning margin is equally small. Values are finally clipped to $[0, 1]$.

Because the probabilities are analytic in z , the interpolation error decreases exponentially in N . The interpolation reproduces constants exactly, so the probabilities of each pixel still sum to one up to rounding.

Only the node probabilities are cached on disk (`generate_ranking_probabilities`), together with the medians and parameters; interpolation happens on every load.

7. Validation (`monte_carlo_check`)

For selected pixels, M voters (default $2 \cdot 10^6$) are sampled from the pixel’s Beta $\times$ Beta distribution, each is ranked directly by sorting distances, and the empirical ranking frequencies are compared with the exact probabilities. The maximum absolute difference should be of the order of the Monte Carlo error $\sqrt{p(1-p)/M} \leq \frac{1}{2\sqrt{M}} \approx 3.5 \cdot 10^{-4}$.

8. Summary of the pipeline

1. For each node median solve Equation 3–Equation 6 for (a, b) (with mirror symmetry and continuation).
2. Build the arrangement of the $\binom{C}{2}$ bisectors in the unit square; each cell has one ranking.
3. For every distinct cell edge, compute $\int \omega$ for all pairs of node parameters (closed form, $u = F(x)$ or $v = G(y)$ substitution, Gauss–Legendre quadrature).
4. Sum signed edge integrals into cell probabilities (Green’s theorem).
5. Interpolate from the $N \times N$ nodes to the $n \times n$ pixels with barycentric Chebyshev interpolation in $\text{logit}(m)$.